Prob 1. (25 points, 5 each) We can better understand the code when we can explain each part easily. Consider the single-line code below for checking if the pushbutton (a key) connected to Pin1 of GPIOA is pressed or not:

```
bool PA1_state = ((GPIOA->IDR & GPIO_PIN_1) == GPIO_PIN_1);
```

1. What is the value of `GPIO_PIN_1`?
2. If the key on PA1 is pressed and the value in IDR corresponding to Pin 1 is 1, what is the value of `GPIOA->IDR & GPIO_PIN_1`?
3. (Continue Part 2.) What is the value of `(GPIOA->IDR & GPIO_PIN_1)== GPIO_PIN_1`?
4. If the key on PA1 is released and the value in IDR corresponding to Pin 1 is 0, what is the value of `GPIOA->IDR & GPIO_PIN_1`?
5. (Continue Part 4.) What is the value of `(GPIOA->IDR & GPIO_PIN_1)== GPIO_PIN_1`?

Prob 2. (15 points.) Consider the problem of estimating the response time for an interrupt. Suppose we need to go the following operations to handle the interrupt with assumptions below:

- It takes the MCU 8 clock cycles for stacking 8 registers. In the meantime, the address of the ISR is fetched.
- We run 10 instructions in the ISR on the MCU which has a 3-stage pipeline. This running is done immediately after the stacking.
- Immediately after the running of the ISR, the MCU uses 8 more cycles to unstack the above registers. Now, we go back the main program.

What is total number of clock cycles that the main program is *interrupted* (suspended) during which the processor executes all the above operations?

Prob 3. (5 points.) Suppose we read the exception number from PSR as 22. What is the IRQn number of the interrupt?

Prob 4. (10 points.) Consider the definition of pointers in Section 8.6.1 of Lctr 8. What are the addresses of the structs for NVIC and SysTick timer, respectively?

Prob 5. (10 points.) Consider the definition of the NVIC struct of Lctr 8. Assume the address of the NVIC struct is at `0xE000_0000`.

- What is the address of ICER[4] (element 4 of array ICER)?
- What is the address of IP[4]?

Prob 6. (10 points.) Consider a certain interrupt which has an IRQn being 69. Assume we need to clear its pending bit.

- Determine which of the ICPRs we need to write to clear its pending bit? (For example ICPR[3])
- What is the location of the bit to write? (For example, bit 22.)

Prob 7. (35 points total, 5 points for the first 3 each and 10 points for the last 2 each) We have learned in Lctr 9 about how to set up the interrupt priority (IP) register using the `NVIC_SetPriority` function. Here we put that knowledge into use. Assume:

- We only use the most significant 4 bits of the byte in each IP register.
- We set up the system to use TWO bits for preemption priority.

Now, we consider the following code to set up the priorities of three interrupt sources, represented by IRQn1, IRQn2, and IRQn3, respectively.

```
(1)  NVIC_SetPriority(IRQn1, 12);
(2)  NVIC_SetPriority(IRQn2, 7);
(3)  NVIC_SetPriority(IRQn3, 3);
```

Answer the following questions:

1. What is the value of `NVIC->IP[IRQn2]` in binary ?
2. Can IRQn1 preempt IRQn2? (Give your reasons.)
3. Can IRQn3 preempt IRQn2? (Give your reasons.)
4. If we **want** a new IRQn4 to be able to preempt IRQn2, what will be the range of priority number we can use when setting it up?
5. If we **do not want** the new IRQn4 to be able to preempt IRQn2, what will be the range of priority number we can use when setting it up?

Prob 8. (20 points, 10 points each) Consider the code in Section 9.4.2 of the class notes of Lctr 9.

```
SYSCFG->EXTICR[0] &= ~MASK_FOR_EXTI0;     // Need to define this mask
SYSCFG->EXTICR[0] |= VALUE_FOR_EXTI0_PA; // Need to define this value
```

Assume we are using `EXTICR1` instead of `EXTICR[0]` and are given the info about EXTICR1 from the reference manual, shown in Figure 1.

9.2.3 SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 1: EXTICR bit fields.

Also assume we want to configure **to use PC3 for EXTI3** instead of PA0 for EXTI0, which is controlled according to the pattern of multiplexer control as shown in Figure 2.

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:12 **EXTI3[3:0]**: EXTI 3 configuration bits
These bits are written by software to select the source input for the EXTI3 external interrupt.
0000: PA[3] pin
0001: PB[3] pin
0010: PC[3] pin
0011: PD[3] pin
0100: PE[3] pin
0101: PF[3] pin
0110: PG[3] pin
0111: PH[3] pin (only on STM32L496xx/4A6xx devices)
1000: PI[3] pin (only for STM32L496xx/4A6xx devices)

Figure 2: Control bits for EXTICR.

Now, the code should be changed to the following:

```
SYSCFG->EXTICR1 &= ~MASK_FOR_EXTI3;     // Need to define this mask
SYSCFG->EXTICR1 |= VALUE_FOR_EXTI3_PC; // Need to define this value
```

Determine the values of the following macros without checking the code:

1. `MASK_FOR_EXTI3`
2. `VALUE_FOR_EXTI3_PC`

Note that if you have difficulty deciphering the above given control, you can refer to Figure 6 of Lctr 7 and the related code.